

pg_basebackup

pg_basebackup – Backup, Restore, and Recovery

What is pg_basebackup?

pg_basebackup is a utility provided by PostgreSQL to take a **physical base backup** of the entire **database cluster**.

Common Use Cases:

- Setting up **standby servers** for streaming replication
- Creating **physical backups** for disaster recovery
- Performing **Point-In-Time Recovery (PITR)**
- Performing **Incremental Backups** ([PostgreSQL v17 feature](#))

It connects to a **running PostgreSQL server** and copies all necessary **data files and WAL segments**, producing a consistent and restorable backup.

----- Backup -----

Pre-requisites parameter and config:

1. User : Requires a user with **replication role** or **superuser** privileges

```
CREATE ROLE repl_user WITH REPLICATION LOGIN ENCRYPTED PASSWORD 'replpass';
```

2. postgresql.conf

- **wal_level=replica (Mandatory)**
- **max_wal_senders >=1 (Mandatory)**
- **archive_mode=on** (Optional if Database in NO Archivelog mode)
- **archive_command = 'cp %p /pgArch/pgsql17/arch/%f'** (Optional if Database in NO Archivelog mode)

3. pg_hba.conf

```
# TYPE      DATABASE          USER            ADDRESS          METHOD
# Local connections for replication (for pg_basebackup run locally)
local      replication    all             trust

# or you can use peer (if same OS user postgres is used)
# local      replication    all             peer

# Remote connections for replication (for pg_basebackup run remotely)
host       replication    repl_user       192.168.2.22/32  scram-sha-256
```

How pg_basebackup Works Internally:

FEATURE	PG_BASEBACKUP SUPPORTS?
Backup of data directory	Yes
Include enough WALs to make backup consistent	Yes
Backup of WAL archives (even with archive mode=on & archive_command configured)	No
Works without archive_mode	Yes

- Performs a **physical file-level backup** by copying the full data directory
- Uses PostgreSQL's **streaming replication protocol**
- Requires a user with **replication role** or **superuser** privileges
- Can be used **while the server is running** (online backup)
- Ensures a transactionally consistent snapshot of the database
- Even with **archive_mode=on** and **archive_command** configured, pg_basebackup **does not back up WAL archive logs** from the archive location. It only includes enough live WAL files to make the backup consistent, streamed from pg_wal/.
- pg_basebackup does not include all files from the pg_wal directory in the backup. Instead, it selectively includes only the WAL files required to make the base backup consistent at the point in time the backup was taken.

What It Includes:

- All essential **data files** of the cluster
- Necessary **WAL (Write-Ahead Log)** segments for recovery
- **Custom tablespaces, replication slots, and large objects**

WAL Handling Options:

- **Default:** WAL files included after backup (-X fetch)
- **Streaming:** WAL files streamed live during backup (-X stream)

How to take Backup using **pg_basebackup** ?

- Want **symlinks preserved** → use (both -Fp & -waldir)-Fp --waldir=/pgWal/pgsql17/wal
- Want **single tar archive** → use -Ft, but **recreate symlinks after restore, for pg_wal**
- The -waldir option in pg_basebackup is supported only when using the plain format (-Fp), not with the tar format (-Ft).

-- Do not use -R here since it's not a replica (No standby).

Typically Backup using: Plain format, Tar format & Tar format (gzip)

1. Take backup in Plain format

```
nohup pg_basebackup -U postgres -D /pgBackups/pgsql17/demo_restore -Fp -Xs
-P -v > pg_basebackup_demo_restore.log 2>&1 &
```

2. Take backup in Tar format

```
nohup pg_basebackup -U postgres -D /pgBackups/pgsql17/demo_tar_backup -Ft -Xs
-P -v > pg_basebackup_tar.log 2>&1 &
```

3. Take backup in Compressed Tar format (gzip)

```
nohup pg_basebackup -U postgres -D /pgBackup/pgsql17/backup/ -Ft -z -Xs -P
-v > pg_basebackup_tar.log 2>&1 &
```

4. Take compressed Tar backup with transfer rate limit

```
nohup pg_basebackup -U postgres -D /pgBackup/pgsql17/backup/ -Ft -z -X
stream -P --max-rate=5M -v > pg_basebackup_tar.log 2>&1 &
```

5. Take compressed Tar backup with server-side gzip compression at max level (9)

```
nohup pg_basebackup -U postgres -D /pgBackup/pgsql17/backup/ -Ft --
compress=server-gzip:9 -Xs -P -v > pg_basebackup.log 2>&1 &
```

6. Take Tar backup from a remote host

```
nohup pg_basebackup -h remote_host -p port -U postgres -D
```

```
/pgBackup/remote_tar_backup -Ft -Xs -P -v > pg_basebackup_remote_tar.log
2>&1 &
```

PG BASE BACKUP FLAG	DESCRIPTION
-U <username>	Specifies the PostgreSQL user to connect as.
-D <directory>	Specifies the target directory for the backup.
-F p	Takes the backup in Plain format (file system copy).
-F t	Takes the backup in Tar archive format.
-Z	Compresses the backup using gzip compression (only valid with tar format).
--compress=server-gzip:<level>	Enables server-side gzip compression with specified compression level (1-9).
-X s	Includes the Write-Ahead Log (WAL) files by copying the WAL segment files.
-X stream	Streams the WAL files while taking the backup for continuous consistency.
-P	Shows progress information during the backup.
--max-rate=<rate>	Limits the maximum transfer rate during the backup (e.g., 5M for 5 megabytes/sec).
-v	-verbose, extra output

----- Restore -----

1. Restore is a File-Level Operation

A base backup is a **physical copy** of the database files — including system catalogs, user data, WAL files, and optionally config files.

Restoring is done by simply copying or extracting the backup files into a valid PostgreSQL data directory (PGDATA) on the target system.

Restore Steps:

1. Stop PostgreSQL (if running)
2. Copy or extract the backup into a clean data directory (PGDATA)
3. Place a file named `recovery.signal` in the data directory
4. Start PostgreSQL to begin recovery

2. Custom Tablespaces Outside PGDATA

- Backups include **symlinks** to external tablespace locations
- On restore:
 - Ensure original paths exist and are accessible
 - Or remap symlinks to new locations
 - Check ownership and permissions (`postgres:postgres, 0700`)

3. Restoring on a Different Host

- Ensure matching directory structure or adjust accordingly
- PostgreSQL **version and architecture** must match
- Update `postgresql.conf` and `pg_hba.conf` as needed

4. Restoring Across PostgreSQL Versions

pg_basebackup is version-specific.

- **Not allowed:** PostgreSQL 14 → PostgreSQL 15 or 17 (Lower to Higher)
- **Not allowed:** PostgreSQL 17 → PostgreSQL 14 (Higher to Lower)
- **Use** `pg_dump/pg_restore` or `pg_upgrade` for version upgrades

----- Recovery -----

How Recovery Works:

1. PostgreSQL detects **recovery.signal** at startup
2. WAL files (in `pg_wal/` or archive) are replayed (Redo apply / Archive apply from `restore_command` location)
3. When recovery completes:
 - PostgreSQL automatically **removes recovery.signal**
 - The server becomes a **primary (read/write)**
4. Recovery stops when:
 - All available WALs are applied, or
 - A recovery target (e.g., timestamp, transaction ID) is reached

What is **recovery.signal** ?

FILE	PURPOSE
<code>recovery.signal</code>	Tells PostgreSQL to enter recovery mode during startup
Automatically removed?	Yes, after recovery completes
Required for standalone restore?	Yes, otherwise WAL replay is skipped

Summary

1. Take a base backup on the source server using 'pg_basebackup'.
2. Take Backup of WAL Archive files separately on Source (Manually, because pg_basebackup does NOT take backup of WAL archive files)
3. Transfer both the base backup and WAL archive logs to the target server.
4. Restore the base backup to the PostgreSQL data directory on the target.
5. Copy the WAL files to the dedicated WAL location (e.g., '/pgwal/pgsql17/wals').
6. Remove '\$PGDATA/pg_wal' and create a symbolic link to the WAL location:
'ln -s /pgwal/pgsql17/wals /pgdata/pgsql17/data/pg_wal'
7. Create an empty '**recovery.signal**' file in '\$PGDATA'.
8. Set '**restore_command**' in 'postgresql.conf' to point to WAL archive backup path (e.g.,
'/pgbackup/pgsql17/backup/wal_archive_bkp').
9. Ensure WAL archive files are restored to '/pgbackup/pgsql17/backup/wal_archive_bkp' for
'restore_command' to access.
10. Update '**tablespace_map**' to reflect correct paths if using custom tablespaces.
11. Start PostgreSQL on the target; recovery will complete and automatically remove 'recovery.signal'.
12. Tablespace symbolic links will create automatically by PostgreSQL.

OPERATION	KEY POINT
Backup	File-level copy using replication protocol
Restore	Place files into PGDATA, handle symlinks, configs, and versions
Recovery	Triggered by <code>recovery.signal</code> , replays WAL, auto-removes signal file

Caution: Your use of any information or materials on this website is entirely at your own risk. It is provided for educational purposes only. It has been tested internally, however, we do not guarantee that it will work for you. Ensure that you run it in your test environment before using.

Thank you,

Rajasekhar Amudala

Email: br8dba@gmail.com

Linkedin: <https://www.linkedin.com/in/rajasekhar-amudala/>